



**UNIVERSITI
MALAYA**

WIA1002 DATA STRUCTURE

PROJECT REPORT

TOPIC 2: SMART LIBRARY

OCC: 12

GROUP 6

LECTURER: Dr. Zainab Malik

NAME	MATRIC NO.
SHAHITYA SHRI A/P SRI KRISHNAH	25006586
TIGITRA JAY C. SELVARAJA	22113137
KAMILIA AHLAM TAQI BINTI TAQI MOHAMED	24001462
WONG XIANG HONG	25060693
LIEVAASHINI A/P THANASEGARAN	25006877

TABLE OF CONTENTS

1. Introduction.....	3
2. Objectives.....	3
3. Problem Statement.....	4
4. Data Structures Used.....	5
4.1. Binary Search Tree (BST).....	5
4.2. Stack.....	6
4.3. Abstract Data Type (ADT).....	7
5. System Design.....	8
5.1. Class Diagram / Structure.....	8
5.2. Program Flowchart.....	9
6. Implementation.....	10
6.1. Add Book Function.....	10
6.2. Search Book Function.....	10
6.2.1. By ISBN.....	10
6.2.2. By Title.....	11
6.3. Borrow Book Function.....	12
6.4. Return Book Function.....	13
6.5. View History Function.....	14
6.6. View Total Book Count Function.....	14
6.7. Add Fine Record Function.....	15
6.8. View Fine Records Function.....	16
6.9. Input Validation.....	16
7. Testing & Output.....	18
8. Challenges.....	23
9. Conclusion.....	24
10. References.....	25

1. Introduction

The Smart Library System is a console-based program developed to simulate the basic operations of a university library. This system allows users to add books, search for books using their ISBN numbers, borrow books, and view their borrowing history through a menu-driven interface. The purpose of this project is to show how different data structures can be applied to solve real-world problems efficiently and effectively.

In this Smart Library project, Binary Search Tree (BST) is implemented to store and organize book records for faster searching operations, while a Stack data structure is implemented to manage borrowing history using the Last-In-First-Out (LIFO) principle. These data structures work together and help improve the efficiency and organization of the whole system.

Moreover, this project applies the concept of Abstract Data Type (ADT) and information hiding through the use of interfaces and object-oriented programming (OOP) principles. Each of our group members was assigned to a specific component of the system, allowing the project to be designed and developed collaboratively and systematically.

Overall, this project helps students to understand the practical implementation of data structures, interfaces, and Object-Oriented Programming (OOP) in Java through the development of a functional Smart Library System.

2. Objectives

- To develop a Smart Library System using a Binary Search Tree (BST) for efficient book storage and searching.
- To implement a Stack data structure to manage borrowing history using the LIFO concept.
- To create a recursive search function to find books quickly by ISBN.
- To apply the concept of Abstract Data Type (ADT) and information hiding in system design.
- To design a user-friendly menu-driven console interface with proper input validation.

3. Problem Statement

A university library stores many books and records borrowing activity every day. The system needs to do two main things well which are finding a book quickly using its ISBN, and showing a student's borrowing history with the most recent book first. These two tasks need different ways of storing data, so one single method cannot handle both well.

To solve this, the project uses two different ways of organising the data. One method stores books by ISBN so they can be found quickly. The other stores the borrowing history so the latest borrowed book shows first, following a Last-In-First-Out (LIFO) order, like a stack of plates where you take the top one off first.

The system also needs to be easy to use for librarians and students through a simple menu, without anyone having to change the code. The inner details of how the data is stored are kept hidden, so users only deal with the menu options such as add a book, find a book, borrow a book, view history, and exit.

4. Data Structures Used

4.1. Binary Search Tree (BST)

The Binary Search Tree (BST) is the primary data structure used in the Smart Library System to manage the library catalogue. Each node in the BST stores information about a book, including its ISBN, title, and author. The ISBN is used as the key value for organizing the tree because it uniquely identifies each book. Following the BST property, books with smaller ISBN values are stored in the left subtree, while books with larger ISBN values are stored in the right subtree.

The BST was chosen because it provides efficient searching, insertion, and deletion operations. In a balanced BST, these operations can be performed in $O(\log n)$ time complexity, which is significantly faster than performing a linear search through an array or list. This makes the BST well-suited for a library system where users frequently need to search for books by ISBN and where new books may be added or removed from the catalogue over time.

In this project, the BST supports key library functions such as adding books, searching for books, and removing books when they are borrowed. The search operation is implemented recursively, allowing the program to efficiently navigate through the tree until the desired ISBN is found. By using a BST as the main catalogue structure, the system is able to organize book records effectively while maintaining fast access to information.

4.2. Stack

The Stack is the data structure used in the Smart Library System to manage the borrowing history. Each item in the stack stores a Book object, which holds the book's ISBN, title, and author. The stack follows a Last-In-First-Out (LIFO) order, which means the most recently borrowed book is always placed on top and is the first one shown when the history is viewed. This matches how borrowing history naturally works, since users usually want to see their most recent activity first.

The Stack was chosen because it provides simple and fast operations for adding and accessing the most recent records. New books are added using the push operation, while pop and peek allow the system to remove or view the book on top of the stack. Since the stack always works from the top, these actions are quick and do not require searching through the whole list, which makes it well-suited for tracking recent borrowing activity where the order of actions matters.

In this project, Stack supports key functions such as adding a borrowed book to the history, displaying the full borrowing history in LIFO order, and returning a book by searching for its ISBN and removing it from the history. An additional feature was also included to count the total number of borrowed books using the stack's built-in size method. By using a Stack to manage borrowing history, the system is able to keep track of user activity accurately while always giving quick access to the most recent records.

4.3. Abstract Data Type (ADT)

In this project, the concept of Abstract Data Type (ADT) was implemented through the use of a Java interface named LibraryADT. The interface was designed to define the main operations that can be carried out in the Smart Library System without exposing the internal implementation details of the system.

The LibraryADT interface contains methods, such as addBook(), searchBook(), borrowBook() and viewLatestHistory(). These methods represent the actions that users can perform within the library system. By using an interface, the program focuses on what the system can do rather than how the operations are implemented internally.

The SmartLibrary class implements the LibraryADT interface and provides the actual functionality for each method. Internal data structures, such as the Binary Search Tree (BST) and Stack are declared as private variables inside the class. This applies the concept of information hiding, where users cannot directly access or edit the internal structures of the system.

In addition, the implementation of ADT improves the organization, modularity and maintainability of the program. Different components of the system can work together efficiently while keeping their implementations separate and secure. Therefore, the use of ADT makes the whole Smart Library System more structured, flexible and easier to manage.

5. System Design

5.1. Class Diagram / Structure

The Smart Library System was designed using Object-Oriented Programming (OOP) concepts to ensure modularity, organization, and maintainability. This system consists of several classes, where each class performs a specific responsibility within the library system.

The LibraryADT interface was designed to define the main operations of the system, including adding books, searching books, and viewing borrowing history. This interface supports abstraction by specifying the behaviours of the system without exposing the implementation details.

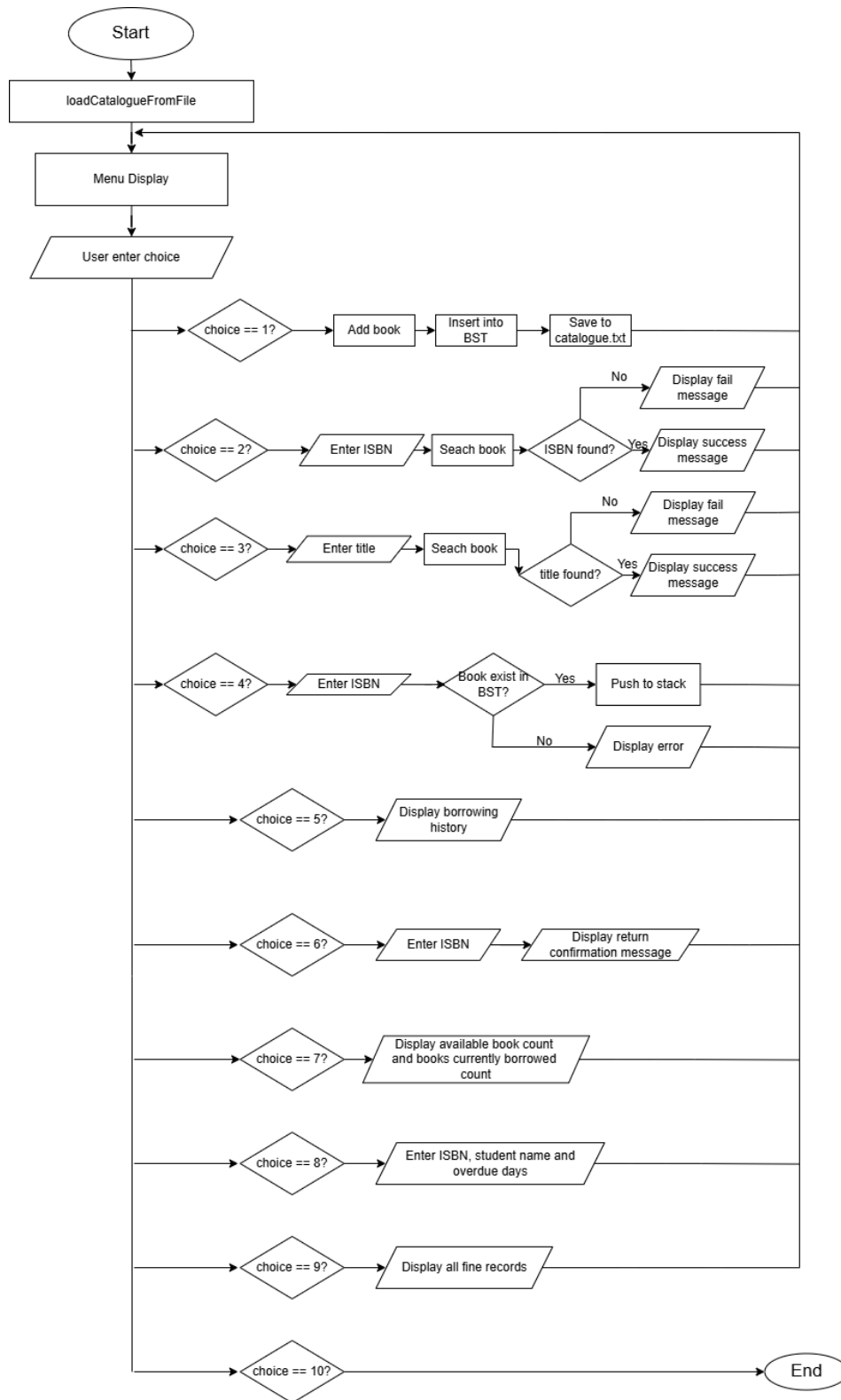
The SmartLibrary class implements the LibraryADT interface and plays a role as the main controller of the system. It connects the different components of the program and manages the interactions between the Binary Search Tree (BST) and Stack data structures.

The BookBST class is responsible for storing and organizing book records using a Binary Search Tree (BST) structure. This allows users to search books efficiently using ISBN numbers. Meanwhile, the BorrowStack class manages the borrowing history of the books implementing the Last-In-First-Out (LIFO) principle.

Moreover, the Book class represents the book entity and stores important information, such as the ISBN number, title, and author name. Getter and setter methods were used to implement encapsulation and information hiding within the system.

Overall, the Smart Library System was designed using Object-Oriented Programming (OOP) principles, allowing each class to carry out a specific role within the system. This improves the system's readability, adaptability and scalability.

5.2. Program Flowchart



6. Implementation

6.1. Add Book Function

```
public class SmartLibrary implements LibraryADT {
    public void handleChoice (int choice, Scanner sc){

        case 1 :    // Add book
            int newIsbn = 0;
            System.out.println (x: "Enter ISBN : ");
            if (sc.hasNextInt()) {
                newIsbn = sc.nextInt();
            } else {
                System.out.println(x: "Invalid ISBN! Please enter numbers only.");
            }
            sc.nextLine();
            System.out.println (x: "Enter title : ");
            String newTitle = sc.nextLine();
            System.out.println (x: "Enter author : ");
            String newAuthor = sc.nextLine();
            addBook(newIsbn, newTitle, newAuthor);
            break;
    }
}
```

Our BST acts as a library catalogue. When the user chooses to add a book, a new book will be added to the BST. The books will be saved in a .txt file before exiting the program.

6.2. Search Book Function

The data lookup services within SmartLibrary application are implemented across two computational paradigms. This architectural split allows users to locate book records using either structural numerical matching (by ISBN) or sequential text properties (by Title).

6.2.1. By ISBN

Locating an inventory record by its unique key employs a localized divide-and-conquer strategy. The task is handled by the `search(int i)` and `searchRec(Book current, int i)` methods within the BookBST class.

```

18      // TASK 3: Recursive Search function (O(log n) efficiency)
19      public Book search(int i) {
20          return searchRec(root, i);
21      }
22
23  ✓ private Book searchRec(Book current, int i) {
24      // Base Case: Book not found or ISBN matches
25      if (current == null || current.getIsbn() == i) {
26          return current;
27      }
28
29      // Recursive Step: Logarithmic traversal
30      if (i < current.getIsbn()) {
31          return searchRec(current.getLeft(), i);
32      }
33      return searchRec(current.getRight(), i);
34  }
35

```

Process Flow:

The search algorithm initiates its traversal at the root of the tree structure. During each recursive step, the system evaluates the target ISBN `i` relative to the key stored in the current node (`current.getIsbn()`). Upon finding a match, the corresponding book object is returned. If the target ISBN is smaller than the current value, the search continues within the left subtree (`current.getLeft()`), effectively pruning the right branch. Conversely, if the target is larger, the operation moves into the right subtree (`current.getRight()`) while disregarding the left branch.

6.2.2. By Title

To support workflows where an operator does not have an item's distinct index, an exhaustive text search method (`searchBookByTitle`) is integrated. Because the catalogue tree structural properties are ordered strictly by numerical values, string parameters do not follow binary partitioning laws.

```

36     public Book searchBookByTitle(String title) {
37         return searchTitleRec(root, title);
38     }
39
40     private Book searchTitleRec(Book current, String title) {
41
42         if (current == null) {
43             return null;
44         }
45
46         if (current.getTitle().equalsIgnoreCase(title)) {
47             return current;
48         }
49
50         Book found = searchTitleRec(current.getLeft(), title);
51
52         if (found != null) {
53             return found;
54         }
55
56         return searchTitleRec(current.getRight(), title);
57     }

```

Process Flow:

The identification of records within this framework utilizes an exhaustive recursive preorder traversal. This algorithm facilitates case-insensitive string matching by executing `.equalsIgnoreCase(title)` against the attribute stored in each node. If a direct match is not found locally, the engine calls itself sequentially down both the left and right branch pointers to inspect every element in the system.

6.3. Borrow Book Function

The Borrow Book Function allows users to borrow a book from the library system. When a book is borrowed, the selected book is removed from the Binary Search Tree (BST) catalogue and added to the borrowing history stack. This helps maintain an updated catalogue while recording the user's borrowing activity.

Process Flow:

1. The user enters the ISBN of the book.
2. The system searches the BST for the matching ISBN.
3. If the book exists:
 - The book record is removed from the catalogue.
 - The book information is pushed into the Stack.
4. A success message is displayed.
5. If the book is not found, an error message is shown.

6.4. Return Book Function

The Return Book Function allows users to return a borrowed book back to the library catalogue. When a book is returned, the system searches for the book inside the borrowing history stack using its ISBN, removes it from the history stack, and restores it back into the Binary Search Tree (BST) catalogue structure.

```
104         case 6 :    // Return book
105             System.out.println("\nEnter ISBN to return : ");
106             if (sc.hasNextInt()) {
107                 int returnIsbn = sc.nextInt();
108                 sc.nextLine();
109                 // Looks for the book in the borrowing history stack
110                 Book b = borrowStack.findAndRemove(returnIsbn);
111
112                 if (b != null) {
113                     // Restores it back to the catalogue BST structure
114                     catalogue.insert(b.getIsbn(), b.getTitle(), b.getAuthor());
115                     System.out.println("\n--- Return Confirmation ---");
116                     System.out.println("ISBN    : " + b.getIsbn());
117                     System.out.println("Title   : " + b.getTitle());
118                     System.out.println("Author  : " + b.getAuthor());
119                     System.out.println("Result  : Book returned successfully!");
120                     System.out.println("Status  : Record restored to Catalogue.");
121                 } else {
122                     System.out.println("Error: This ISBN was not found in the Borrowed History.");
123                 }
124                 System.out.println("-----");
125             } else {
126                 System.out.println("Invalid ISBN! Please enter numbers only.");
127                 sc.nextLine();
128             }
129             break;
```

Process Flow:

- a. The user inputs the ISBN of the book they wish to return.
- b. The system invokes `borrowStack.findAndRemove(returnIsbn)` to search through the elements inside the dynamic stack structure.
- c. If the matching book is found in the borrowing history:
 - The book node is removed from the history stack.
 - The book attributes (ISBN, title, and author) are passed to `catalogue.insert()`, restoring the record safely back into the BST catalogue.
 - A successful return confirmation message block is printed on the screen.
- d. If the book does not exist in the history stack, an error message is displayed.

6.5. View History Function

```
//SHOW
public void show(){

    //Check if stack is empty
    if(stack.isEmpty()){
        System.out.println("History is empty.");
    }

    else{
        System.out.println("\n--- Borrowing History ---");

        // Stack from top of stack
        for (int i = stack.size() - 1; i >= 0; i--){

            Book b = stack.get(i);

            System.out.println ("[ISBN: " + b.getIsbn() + "] " + b.getTitle() + " by " + b.getAuthor());
        }
        System.out.println("-----");
    }
}
```

The View Borrowing History Function shows the user a list of all the books that have been borrowed. The books are displayed in Last-In-First-Out (LIFO) order, which means the most recently borrowed book appears first and the oldest one appears last.

Process Flow:

- a) The user selects the View History option from the main menu.
- b) The system calls the show() method to display the borrowing history.
- c) The method first checks if the stack is empty. If there are no borrowed books, it prints "History is empty." and returns to the menu.
- d) If the stack is not empty, the system prints a "Borrowing History" heading and loops through the stack starting from the top (the most recent book) down to the bottom (the oldest book).
- e) For each book, the system displays its ISBN, title, and author, then returns to the main menu once the full list has been shown.

6.6. View Total Book Count Function

The View Total Book Count Function displays live library operational statistics to the user. It shows the total count of available books remaining in the catalog alongside the total count of books currently checked out by users.

```

131         case 7 :    // Extra feature: View Total Book Count
132             System.out.println("");
133             System.out.println("--- Library Statistics ---");
134             System.out.println("Available Books in Catalogue   : " + catalogue.getBookCount());
135             System.out.println("Total Books Currently Borrowed : " + borrowStack.getBorrowedCount());
136             System.out.println("-----");
137             break;
---
```

Process Flow:

- The user selects option 7 from the main menu console interface.
- The system prints a structured "Library Statistics" information window.
- To calculate available books, the system invokes `catalogue.getBookCount()`, which runs a recursive traversal method (`countNodes`) over the BST structure, adding up every node present in the tree.
- To calculate borrowed books, the system invokes `borrowStack.getBorrowedCount()`, which returns the current structural size of the internal stack framework.
- The combined numeric balances are neatly displayed to the operator before returning to the main menu loop.

6.7. Add Fine Record Function

```

public class FineManager {

    public void addFine(int isbn, String studentName, int overdueDays) {
        Fine fine = new Fine(isbn, studentName, overdueDays);
        fines.add(fine);
    }

}

case 8 :    // Extra feature: Add fine record
System.out.println(x: "\n--- Add Fine Record ---");
System.out.print(s: "Enter ISBN: ");
int isbn = sc.nextInt();
sc.nextLine();
System.out.print(s: "Enter Student Name: ");
String studentName = sc.nextLine();
System.out.print(s: "Enter Overdue Days: ");
int overdueDays = sc.nextInt();
sc.nextLine();
fineManager.addFine(isbn, studentName, overdueDays);
System.out.println(x: "Fine record added successfully!");
break;
```

Process flow:

Fine Manager

In `FineManager` class, a method `addFine` was created. This method receives `isbn`, `studentName` and `overdueDays` as the parameters. Then, it creates a new `Fine` and adds it to the list of fines.

Main

In Main, when case 8 is chosen, user is required to input the ISBN, Student Name and Overdue Days. Then, method addFine is executed.

6.8. View Fine Records Function

```
public class FineManager {  
    public void displayFines() {  
        System.out.println(x: "\n--- Fine Records ---");  
        if (fines.isEmpty()) {  
            System.out.println(x: "No fines recorded.");  
        } else {  
            for (Fine fine : fines) {  
                fine.displayFine();  
                System.out.println(x: "-----");  
            }  
        }  
    }  
}  
  
case 9 :    // Extra feature: View fine records  
    fineManager.displayFines();  
    break;
```

View fine displays all fines contained in the list. The list is saved in fine.txt to ensure it does not go missing after program execution.

6.9. Input Validation

Input validation is implemented to ensure that users enter valid and correct input into the system. This helps prevent system errors, improves reliability, and ensures smooth operation of the Smart Library System.

1. ISBN Validation

If the user enters invalid input such as letters or symbols, the system displays an error message and requests the user to enter the ISBN again.

```
--- SmartLibrary Menu ---  
1. Add Book  
2. Search (BST)  
3. Borrow (Stack)  
4. History  
5. Exit  
Choice :  
2  
Enter ISBN :  
abc  
Invalid ISBN! Please enter numbers only.
```

2. Empty Search Result Validation

If the book is not found, the system informs the user instead of crashing or returning incorrect data.


```
--- SmartLibrary Menu ---  
1. Add Book  
2. Search (BST)  
3. Borrow (Stack)  
4. History  
5. Exit  
Choice :  
2  
Enter ISBN :  
999  
Book not found.
```

3. Menu Choice Validation

If the user enters an invalid menu option, an error message is displayed.

```
--- SmartLibrary Menu ---  
1. Add Book  
2. Search (BST)  
3. Borrow (Stack)  
4. History  
5. Exit  
Choice :  
99  
Invalid input. Please try again
```

7. Testing & Output

Test Case	Screenshots	Explanation
Add Book	<pre>--- SmartLibrary Menu --- 1. Add New Book 2. Search Book (BST) 3. Search Book By Title 4. Borrow Book (Stack) 5. View History 6. Return Book 7. View Total Book Count 8. Add Fine Record 9. View Fines 10. Exit Choice : 1 Enter ISBN : 1022 Enter title : I want to eat tteobokki Enter author : Sam Miller --- Book Registration --- Result: Book added successfully!</pre>	The user added the book successfully.
Search Book	<pre>--- SmartLibrary Menu --- 1. Add New Book 2. Search Book (BST) 3. Search Book By Title 4. Borrow Book (Stack) 5. View History 6. Return Book 7. View Total Book Count 8. Add Fine Record 9. View Fines 10. Exit Choice : 2 Enter ISBN : 1022 --- Search Results --- Found: [ISBN: 1022] I want to eat tteobokki by Sam Miller</pre>	The user found the desired book by ISBN search.

Search Non-existing Book	<pre> --- SmartLibrary Menu --- 1. Add New Book 2. Search Book (BST) 3. Search Book By Title 4. Borrow Book (Stack) 5. View History 6. Return Book 7. View Total Book Count 8. Add Fine Record 9. View Fines 10. Exit Choice : 2 Enter ISBN : 1023 --- Search Results --- Result: Book not found. </pre>	The user couldn't find the desired book by ISBN.
Search Book by Title	<pre> --- SmartLibrary Menu --- 1. Add New Book 2. Search Book (BST) 3. Search Book By Title 4. Borrow Book (Stack) 5. View History 6. Return Book 7. View Total Book Count 8. Add Fine Record 9. View Fines 10. Exit Choice : 3 Enter book title: Introduction to Systems Book found! ISBN: 1001 Title: Introduction to Systems Author: Ada Lovelace </pre>	The user found the desired book by title search.

Borrow Book	<pre> --- SmartLibrary Menu --- 1. Add New Book 2. Search Book (BST) 3. Search Book By Title 4. Borrow Book (Stack) 5. View History 6. Return Book 7. View Total Book Count 8. Add Fine Record 9. View Fines 10. Exit Choice : 4 Enter ISBN : 1021 --- Borrow Transaction --- Streets of George Town added to borrowing history You have successfully borrowed book : 1021, Streets of George Town </pre>	The user has successfully borrowed a book and the borrowed book has been pushed to a stack.
View History	<pre> --- SmartLibrary Menu --- 1. Add New Book 2. Search Book (BST) 3. Search Book By Title 4. Borrow Book (Stack) 5. View History 6. Return Book 7. View Total Book Count 8. Add Fine Record 9. View Fines 10. Exit Choice : 5 --- Borrowing History --- [ISBN: 1021] Streets of George Town by Khoo Su Nin </pre>	Checking borrowing history.
Return Book	<pre> --- SmartLibrary Menu --- 1. Add New Book 2. Search Book (BST) 3. Search Book By Title 4. Borrow Book (Stack) 5. View History 6. Return Book 7. View Total Book Count 8. Add Fine Record 9. View Fines 10. Exit Choice : 6 Enter ISBN to return : 1021 --- Return Confirmation --- ISBN : 1021 Title : Streets of George Town Author : Khoo Su Nin Result : Book returned successfully! Status : Record restored to Catalogue. </pre>	The user successfully returned the borrowed book by entering its ISBN, and the record was restored to the catalogue.

View Total Book Count	<pre> --- SmartLibrary Menu --- 1. Add New Book 2. Search Book (BST) 3. Search Book By Title 4. Borrow Book (Stack) 5. View History 6. Return Book 7. View Total Book Count 8. Add Fine Record 9. View Fines 10. Exit Choice : 7 --- Library Statistics --- Available Books in Catalogue : 22 Total Books Currently Borrowed : 0 </pre>	The user viewed the live library statistics showing the total available books and currently borrowed books.
Add Fine Record	<pre> --- SmartLibrary Menu --- 1. Add New Book 2. Search Book (BST) 3. Search Book By Title 4. Borrow Book (Stack) 5. View History 6. Return Book 7. View Total Book Count 8. Add Fine Record 9. View Fines 10. Exit Choice : 8 --- Add Fine Record --- Enter ISBN: 1019 Enter Student Name: Hani Enter Overdue Days: 3 Fine record added successfully! </pre>	The user successfully added a new student fine record.

View Fines	<pre>--- SmartLibrary Menu --- 1. Add New Book 2. Search Book (BST) 3. Search Book By Title 4. Borrow Book (Stack) 5. View History 6. Return Book 7. View Total Book Count 8. Add Fine Record 9. View Fines 10. Exit Choice : 9 --- Fine Records --- ISBN: 1021 Student Name: Ali bin Abu Overdue Days: 3 Fine Amount: RM1.50 ----- ISBN: 1007 Student Name: Cheong Li Han Overdue Days: 4 Fine Amount: RM2.00 ----- ISBN: 1010 Student Name: Balqis binti Mohd Alif Overdue Days: 4 Fine Amount: RM2.00 -----</pre>	The user successfully displayed the complete list of all recorded student fines and overdue balances.
------------	---	---

8. Challenges

During the development of the Smart Library System, several challenges were encountered. One of the main difficulties was implementing the Binary Search Tree (BST) structure correctly. Since the BST serves as the primary catalogue for storing books, careful attention was required when designing the insertion, searching, and deletion operations. In particular, removing a book from the BST after it was borrowed proved challenging because different cases had to be handled, such as nodes with no children, one child, or two children.

Another challenge was integrating multiple data structures within a single system. The project required the BST to manage book records while a Stack was used to maintain borrowing history. Ensuring that books were correctly transferred from the catalogue to the borrowing history without causing inconsistencies required thorough testing and debugging.

File management was also a significant challenge. The team wanted the library records to persist even after the program was closed. This required designing a file-handling mechanism to load book records from a text file at startup and save the updated catalogue when the program exited. Deciding whether data should be written immediately after every operation or only when the program terminated required careful consideration of efficiency and system design.

Additionally, designing a user-friendly console interface was challenging. The system needed to validate user input and handle invalid entries gracefully to prevent runtime errors. Ensuring that the menu system remained intuitive while supporting all required operations took several iterations.

Finally, collaboration among team members required coordination because different components, such as the BST, Stack, file management, and interface design, were developed separately. Integrating these components into a single functional application required consistent coding practices, regular communication, and extensive testing to ensure compatibility between modules.

9. Conclusion

The development of our console-based Smart Library System successfully demonstrates how basic data structures and Object-Oriented Programming (OOP) concepts can be used to solve real-world problems. Instead of using simple static arrays, our group developed a more organized and efficient system that can handle library operations smoothly without system crashes.

Through good teamwork, we divided the system into different components and applied suitable data structures for each function:

- **Binary Search Tree (BST):** The `BookBST` class is used to manage the main library catalog and allows faster searching of books using the ISBN number.
- **Stack Structure:** The `BorrowStack` class stores borrowing history based on the Last-In-First-Out (LIFO) concept.
- **ArrayList Dynamic Collection:** The `FineManager` class is used to record and calculate student late fees dynamically.

Besides that, by using a Java interface (`LibraryADT`) and private variables, we successfully applied important programming concepts such as Abstract Data Type (ADT), encapsulation, and information hiding. During the testing phase, we also added string cleaning filters in the text file reader to remove hidden file signatures. This helped prevent data loading errors and improved the reliability of the system.

Overall, this project helped us better understand how selecting the correct algorithms and data structures can improve software performance and organization. The final Smart Library System is more stable, well-structured, and easier to maintain while successfully achieving all the assignment requirements.

10. References

Athukorala. (n.d.). *GitHub - Athukorala99/SmartLibrary: Library Management System & Website! This project provides a comprehensive solution for managing library operations through a desktop application for librarians and a website for students.* GitHub. <https://github.com/Athukorala99/SmartLibrary>

GeeksforGeeks. (2016, February 4). *Stack Class in Java. This article explains the concepts, implementation, and operations of the Stack class in Java, including the Last In First Out (LIFO) principle and common methods such as push(), pop(), peek(), and search() used for managing elements efficiently.* GeeksforGeeks. <https://www.geeksforgeeks.org/java/stack-class-in-java/>

GeeksforGeeks. (2025). *Binary Search Tree Data Structure. This article explains the concepts, implementation, and operations of Binary Search Trees (BST), including insertion and searching techniques used for efficient data retrieval in applications.* GeeksforGeeks. <https://www.geeksforgeeks.org/binary-search-tree-data-structure/>

GeeksforGeeks. (2017, September 11). *Abstract Data Types.* GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/abstract-data-types/>

Hitenchandra. (2025, August 22). *Building a Library Management System in Java Using OOP Principles.* Medium.com. <https://medium.com/@hitenchandra/building-a-library-management-system-in-java-using-oop-principles-3695e4a7cecb>